

Chapter 29 — Shortest Path Algorithms

0. What You Will Learn in This Chapter

Shortest path problems ask for the minimum cost, distance, time, or number of steps needed to reach a target.

Different shortest path algorithms apply depending on edge weights:

- **BFS** for unweighted graphs
- **Dijkstra** for non-negative weighted graphs
- **0-1 BFS** for edge weights 0 or 1
- **Bellman-Ford** for graphs with negative edges
- **Floyd-Warshall** for all-pairs shortest paths
- **Topological DP** for shortest paths in DAGs

By the end of this chapter, you should understand:

- How to identify shortest path problems
- When BFS is enough
- When Dijkstra is needed
- How priority queue Dijkstra works
- How to handle stale priority queue entries
- Why Dijkstra fails with negative weights
- How 0-1 BFS works
- How Bellman-Ford works
- How Floyd-Warshall works
- How to solve network delay problems
- How to solve cheapest flights with stops
- How to solve minimum effort path
- How to solve path with probabilities
- How to choose the right shortest path algorithm

Core problems covered in this chapter:

1. Network Delay Time
 2. Path With Minimum Effort
 3. Cheapest Flights Within K Stops
 4. Minimum Cost to Make at Least One Valid Path in a Grid
 5. Swim in Rising Water
 6. Find the City With the Smallest Number of Neighbors
 7. Path With Maximum Probability
 8. Min Cost to Connect All Points, contrast with MST
 9. Shortest Path in Binary Matrix, BFS contrast
 10. Shortest Path in a Grid with Obstacles Elimination, BFS state contrast
 11. Bellman-Ford template
 12. Floyd-Warshall template
 13. 0-1 BFS template
 14. Dijkstra on grid
 15. Dijkstra with state
 16. Shortest path in DAG
 17. Reconstruct shortest path
 18. Count shortest paths
-

1. What Is a Shortest Path Problem?

A shortest path problem asks:

What is the minimum cost to get from one node to another?

Cost can mean:

- number of edges
- time
- distance
- effort
- price
- risk
- probability transformed into cost
- maximum edge along path
- number of changes

Example:

0 --5-- 1

0 --2-- 2

2 --1-- 1

Shortest path from 0 to 1 is:

0 -> 2 -> 1

Cost:

3

not direct edge cost 5.

2. Why Multiple Algorithms Exist

Not all shortest path problems are the same.

The right algorithm depends on edge weights.

Unweighted	BFS
All weights non-negative	Dijkstra
Weights only 0 or 1	0-1 BFS
Negative edges possible	Bellman-Ford
All pairs shortest path	Floyd-Warshall
DAG	Topological DP

Choosing the wrong algorithm can produce incorrect answers or time out.

3. BFS vs Dijkstra

3.1 BFS

Use BFS when every edge has equal cost.

Example:

each move costs 1

BFS explores by number of edges.

3.2 Dijkstra

Use Dijkstra when edges have different non-negative weights.

Example:

road times: 5, 20, 3

Dijkstra explores nodes by smallest known distance first.

4. Core Need / Why Use Dijkstra?

BFS uses a normal queue.

But with weighted edges, reaching a node in fewer edges may not mean lower cost.

Example:

0 -> 1 cost 100

0 -> 2 cost 1

2 -> 1 cost 1

BFS may reach 1 directly first with cost 100.

But shortest cost is 2 through 2.

Dijkstra fixes this by always processing the currently cheapest known state first.

It uses:

priority_queue

as a min-heap.

5. When to Use Each Algorithm

5.1 Use BFS

When:

- graph is unweighted
- every move costs 1
- shortest number of steps

Examples:

- Shortest Path in Binary Matrix
- Word Ladder
- Open the Lock

5.2 Use Dijkstra

When:

- edge weights are non-negative

- need shortest cost/time/distance
- graph may be sparse

Examples:

- Network Delay Time
- Path With Minimum Effort
- Path With Maximum Probability
- Swim in Rising Water

5.3 Use 0-1 BFS

When:

- edge weights are only 0 or 1

Examples:

- Minimum Cost to Make at Least One Valid Path in a Grid
- edge reversal minimum cost problems

5.4 Use Bellman-Ford

When:

- negative edge weights exist
- need detect negative cycle
- constraints are moderate
- stops/edge-limit constraints resemble repeated relaxation

Examples:

- Cheapest Flights Within K Stops
- currency arbitrage negative-cycle concept

5.5 Use Floyd-Warshall

When:

- need shortest paths between all pairs
- n is small
- graph is dense

Examples:

- Find the City With the Smallest Number of Neighbors
- all-pairs reachability

5.6 Use Topological DP

When:

- graph is DAG
 - edges can be negative
 - need shortest/longest path in DAG
-

6. When NOT to Use Dijkstra

Do not use standard Dijkstra when:

6.1 Negative Edges Exist

Dijkstra assumes once the smallest-distance node is finalized, it cannot improve. Negative edges break this assumption.

6.2 All Edges Equal Cost

BFS is simpler and faster.

6.3 Need All-Pairs and n Is Small

Floyd-Warshall may be simpler.

6.4 Need MST

Minimum spanning tree is different from shortest path.
Use Kruskal or Prim.

7. Mental Models

7.1 BFS as Equal-Cost Dijkstra

BFS is Dijkstra where every edge cost is 1.

7.2 Dijkstra as Cheapest Frontier First

Always expand the currently cheapest frontier state.

7.3 Relaxation

If going through node u improves distance to v , update it.

```
if (dist[u] + weight < dist[v]) {  
    dist[v] = dist[u] + weight;  
}
```

This is called relaxation.

7.4 Stale Heap Entries

Priority queue may contain old worse distances.

Skip them:

```
if (currentDist > dist[node]) {  
    continue;  
}
```

8. Pattern Recognition Checklist

Ask:

19. Is the problem asking minimum cost/time/distance?
 20. Are edges weighted?
 21. Are weights non-negative?
 22. Are weights 0/1 only?
 23. Are negative weights possible?
 24. Is graph unweighted?
 25. Is there a stops/edge-limit constraint?
 26. Need shortest paths from one source or all pairs?
 27. Is graph a DAG?
 28. Does state need extra metadata?
-

9. C++ Tools Needed

9.1 Weighted Adjacency List

```
vector<vector<pair<int, int>>> graph(n);
```

Each pair:

```
neighbor, weight
```

9.2 Min-Heap Priority Queue

C++ priority queue is max-heap by default.

Use:

```
priority_queue<pair<int, int>,  
              vector<pair<int, int>>,  
              greater<pair<int, int>>> pq;
```

Pair:

distance, node

9.3 Distance Array

```
vector<int> dist(n, INF);
```

Use large INF:

```
const int INF = 1e9;
```

For large sums:

```
const long long INF = 4e18;
```

9.4 Deque for 0-1 BFS

```
deque<int> dq;
```

Push front for 0-cost edge.

Push back for 1-cost edge.

10. Core Templates

Template 1 — Dijkstra

```
vector<int> dijkstra(int n,  
                   vector<vector<pair<int, int>>>& graph,  
                   int source) {  
    const int INF = 1e9;  
  
    vector<int> dist(n, INF);  
    dist[source] = 0;  
  
    priority_queue<pair<int, int>,  
                  vector<pair<int, int>>,  
                  greater<pair<int, int>>> pq;  
  
    pq.push({0, source});  
  
    while (!pq.empty()) {  
        auto [currentDist, node] = pq.top();
```

```

    pq.pop();

    if (currentDist > dist[node]) {
        continue;
    }

    for (auto [neighbor, weight] : graph[node]) {
        int nextDist = currentDist + weight;

        if (nextDist < dist[neighbor]) {
            dist[neighbor] = nextDist;
            pq.push({nextDist, neighbor});
        }
    }
}

return dist;
}

```

Template 2 — Dijkstra on Grid

```

vector<vector<int>> dist(rows, vector<int>(cols, INF));

priority_queue<tuple<int, int, int>,
               vector<tuple<int, int, int>>,
               greater<tuple<int, int, int>>> pq;

dist[0][0] = 0;
pq.push({0, 0, 0});

while (!pq.empty()) {
    auto [cost, r, c] = pq.top();
    pq.pop();

    if (cost > dist[r][c]) {
        continue;
    }

    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;

        if (invalid) {

```

```

        continue;
    }

    int nextCost = ...;

    if (nextCost < dist[nr][nc]) {
        dist[nr][nc] = nextCost;
        pq.push({nextCost, nr, nc});
    }
}
}
}

```

Template 3 — 0-1 BFS

```

vector<int> zeroOneBFS(int n,
                      vector<vector<pair<int, int>>>& graph,
                      int source) {
    const int INF = 1e9;

    vector<int> dist(n, INF);
    deque<int> dq;

    dist[source] = 0;
    dq.push_front(source);

    while (!dq.empty()) {
        int node = dq.front();
        dq.pop_front();

        for (auto [neighbor, weight] : graph[node]) {
            int nextDist = dist[node] + weight;

            if (nextDist < dist[neighbor]) {
                dist[neighbor] = nextDist;

                if (weight == 0) {
                    dq.push_front(neighbor);
                } else {
                    dq.push_back(neighbor);
                }
            }
        }
    }
}

```

```
    return dist;
}
```

Template 4 — Bellman-Ford

```
vector<int> bellmanFord(int n, vector<vector<int>>& edges, int source) {
    const int INF = 1e9;

    vector<int> dist(n, INF);
    dist[source] = 0;

    for (int i = 0; i < n - 1; i++) {
        vector<int> nextDist = dist;

        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];
            int w = edge[2];

            if (dist[u] == INF) {
                continue;
            }

            if (dist[u] + w < nextDist[v]) {
                nextDist[v] = dist[u] + w;
            }
        }

        dist = nextDist;
    }

    return dist;
}
```

Note:

Using copy nextDist limits paths by number of edges per iteration.

For classic Bellman-Ford without edge limit, relax in-place.

Template 5 — Floyd-Warshall

```
vector<vector<int>> floydWarshall(int n, vector<vector<int>>& edges) {
    const int INF = 1e9;
```

```

vector<vector<int>> dist(n, vector<int>(n, INF));

for (int i = 0; i < n; i++) {
    dist[i][i] = 0;
}

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];
    int w = edge[2];

    dist[u][v] = min(dist[u][v], w);
    dist[v][u] = min(dist[v][u], w); // if undirected
}

for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] == INF || dist[k][j] == INF) {
                continue;
            }

            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}

return dist;
}

```

Template 6 — Shortest Path Reconstruction

```

vector<int> parent(n, -1);

if (dist[u] + w < dist[v]) {
    dist[v] = dist[u] + w;
    parent[v] = u;
}

```

Reconstruct:

```

vector<int> path;

for (int cur = target; cur != -1; cur = parent[cur]) {

```

```
    path.push_back(cur);  
}  
  
reverse(path.begin(), path.end());
```

11. Algorithm Decision Table

unweighted shortest path	BFS
grid shortest path with equal moves	BFS
weighted non-negative graph	Dijkstra
edge weights 0 or 1	0-1 BFS
negative weights	Bellman-Ford
all-pairs shortest path	Floyd-Warshall
shortest path in DAG	topological DP
max probability path	Dijkstra with max-heap
minimize maximum edge	Dijkstra variant / binary search + BFS
stops constraint	Bellman-Ford style / BFS state

12. Full Problem Walkthrough 1 — LeetCode 743: Network Delay Time

12.1 Problem Summary

Given directed weighted edges:

u, v, w

Signal starts at node k.

Return time for all nodes to receive signal.

If impossible, return -1.

Nodes are 1-indexed.

12.2 How to Identify Dijkstra

Need shortest time from one source to all nodes.

Weights are non-negative.

Use Dijkstra.

12.3 C++ Code

```
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int, int>>> graph(n + 1);

        for (auto& edge : times) {
            int u = edge[0];
            int v = edge[1];
            int w = edge[2];

            graph[u].push_back({v, w});
        }

        const int INF = 1e9;
        vector<int> dist(n + 1, INF);
        dist[k] = 0;

        priority_queue<pair<int, int>,
                      vector<pair<int, int>>,
                      greater<pair<int, int>>> pq;

        pq.push({0, k});

        while (!pq.empty()) {
            auto [currentDist, node] = pq.top();
            pq.pop();

            if (currentDist > dist[node]) {
                continue;
            }

            for (auto [neighbor, weight] : graph[node]) {
                int nextDist = currentDist + weight;

                if (nextDist < dist[neighbor]) {
                    dist[neighbor] = nextDist;
                    pq.push({nextDist, neighbor});
                }
            }
        }
    }
}
```

```
int answer = 0;

for (int node = 1; node <= n; node++) {
    if (dist[node] == INF) {
        return -1;
    }

    answer = max(answer, dist[node]);
}

return answer;
};
```

12.4 Complexity

Time:

$O((V + E) \log V)$

Space:

$O(V + E)$

12.5 Common Mistakes

- Forgetting nodes are 1-indexed
- Building undirected graph when edges are directed
- Returning distance to only one target
- Not skipping stale heap entries

13. Full Problem Walkthrough 2 — LeetCode 1631: Path With Minimum Effort

13.1 Problem Summary

Given heights grid.

Path effort is maximum absolute height difference between adjacent cells along path.
Return minimum possible effort from top-left to bottom-right.

13.2 How to Identify Dijkstra Variant

Path cost is not sum.

Path cost is maximum edge used.

But it is still monotonic:

$\text{new effort} = \max(\text{current effort}, \text{edge effort})$

13.3 C++ Code

```
class Solution {
public:
    int minimumEffortPath(vector<vector<int>>& heights) {
        int rows = heights.size();
        int cols = heights[0].size();

        const int INF = 1e9;

        vector<vector<int>> dist(rows, vector<int>(cols, INF));
        dist[0][0] = 0;

        priority_queue<tuple<int, int, int>,
                      vector<tuple<int, int, int>>,
                      greater<tuple<int, int, int>>> pq;

        pq.push({0, 0, 0});

        vector<pair<int, int>> dirs = {
            {1, 0},
            {-1, 0},
            {0, 1},
            {0, -1}
        };

        while (!pq.empty()) {
            auto [effort, r, c] = pq.top();
            pq.pop();

            if (r == rows - 1 && c == cols - 1) {
                return effort;
            }

            if (effort > dist[r][c]) {
                continue;
            }

            for (auto [dr, dc] : dirs) {
                int nr = r + dr;
                int nc = c + dc;
```

```

        if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
            continue;
        }

        int edgeEffort = abs(heights[r][c] - heights[nr][nc]);
        int nextEffort = max(effort, edgeEffort);

        if (nextEffort < dist[nr][nc]) {
            dist[nr][nc] = nextEffort;
            pq.push({nextEffort, nr, nc});
        }
    }
}

return 0;
}
};

```

13.4 Complexity

Time:

$O(R * C * \log(R * C))$

Space:

$O(R * C)$

13.5 Alternative

Binary search effort + BFS/DFS reachability also works.

14. Full Problem Walkthrough 3 — LeetCode 787: Cheapest Flights Within K Stops

14.1 Problem Summary

Given flights:

from, to, price

Find cheapest price from source to destination with at most k stops.

At most k stops means at most:

k + 1 edges

14.2 Why Normal Dijkstra Is Tricky

State includes:

node + stops used

A cheaper path to a node with too many stops may be useless.

A more expensive path with fewer stops may be better later.

A Bellman-Ford style relaxation for $k + 1$ edges is straightforward.

14.3 C++ Code — Bellman-Ford Style

```
class Solution {
public:
    int findCheapestPrice(int n,
                        vector<vector<int>>& flights,
                        int src,
                        int dst,
                        int k) {
        const int INF = 1e9;

        vector<int> dist(n, INF);
        dist[src] = 0;

        for (int i = 0; i <= k; i++) {
            vector<int> nextDist = dist;

            for (auto& flight : flights) {
                int u = flight[0];
                int v = flight[1];
                int price = flight[2];

                if (dist[u] == INF) {
                    continue;
                }

                nextDist[v] = min(nextDist[v], dist[u] + price);
            }

            dist = nextDist;
        }

        return dist[dst] == INF ? -1 : dist[dst];
    }
};
```

14.4 Why Copy nextDist?

Each iteration represents using one more edge.

Using a copy prevents using updated values from the same iteration and accidentally taking too many edges.

14.5 Complexity

Let:

$E = \text{flights.size()}$

Time:

$O((k + 1) * E)$

Space:

$O(n)$

15. Full Problem Walkthrough 4 — LeetCode 1368: Minimum Cost to Make at Least One Valid Path in a Grid

15.1 Problem Summary

Each grid cell has direction:

1 = right

2 = left

3 = down

4 = up

Following the direction costs 0.

Changing direction costs 1.

Find minimum cost path from top-left to bottom-right.

15.2 How to Identify 0-1 BFS

Every move has cost either:

0 or 1

Use 0-1 BFS with deque.

15.3 C++ Code

```
class Solution {  
public:
```

```

int minCost(vector<vector<int>>& grid) {
    int rows = grid.size();
    int cols = grid[0].size();

    vector<pair<int, int>> dirs = {
        {0, 1}, // 1 right
        {0, -1}, // 2 left
        {1, 0}, // 3 down
        {-1, 0} // 4 up
    };

    const int INF = 1e9;
    vector<vector<int>> dist(rows, vector<int>(cols, INF));

    deque<pair<int, int>> dq;
    dq.push_front({0, 0});
    dist[0][0] = 0;

    while (!dq.empty()) {
        auto [r, c] = dq.front();
        dq.pop_front();

        for (int i = 0; i < 4; i++) {
            int nr = r + dirs[i].first;
            int nc = c + dirs[i].second;

            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
                continue;
            }

            int cost = (grid[r][c] == i + 1) ? 0 : 1;
            int nextDist = dist[r][c] + cost;

            if (nextDist < dist[nr][nc]) {
                dist[nr][nc] = nextDist;

                if (cost == 0) {
                    dq.push_front({nr, nc});
                } else {
                    dq.push_back({nr, nc});
                }
            }
        }
    }
}

```

```

    }
}

return dist[rows - 1][cols - 1];
}
};

```

15.4 Complexity

Time:

$O(R * C)$

Space:

$O(R * C)$

15.5 Why Not Normal BFS?

Moves do not all have same cost.

Some moves cost 0, some cost 1.

Normal BFS would treat them equally and may be wrong.

16. Full Problem Walkthrough 5 — LeetCode 778: Swim in Rising Water

16.1 Problem Summary

Grid values represent elevation.

At time t , you can enter cells with value $\leq t$.

Return minimum time to reach bottom-right.

16.2 Dijkstra Interpretation

Path cost is maximum elevation encountered along path.

Relaxation:

```
nextCost = max(currentCost, grid[nr][nc])
```

Same style as minimum effort path.

16.3 C++ Code

```

class Solution {
public:
    int swimInWater(vector<vector<int>>& grid) {
        int n = grid.size();

```

```
vector<vector<int>> dist(n, vector<int>(n, INT_MAX));
dist[0][0] = grid[0][0];
```

```
priority_queue<tuple<int, int, int>,
               vector<tuple<int, int, int>>,
               greater<tuple<int, int, int>>> pq;
```

```
pq.push({grid[0][0], 0, 0});
```

```
vector<pair<int, int>> dirs = {
    {1, 0},
    {-1, 0},
    {0, 1},
    {0, -1}
};
```

```
while (!pq.empty()) {
    auto [time, r, c] = pq.top();
    pq.pop();
```

```
    if (r == n - 1 && c == n - 1) {
        return time;
    }
```

```
    if (time > dist[r][c]) {
        continue;
    }
```

```
    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;
```

```
        if (nr < 0 || nc < 0 || nr >= n || nc >= n) {
            continue;
        }
```

```
        int nextTime = max(time, grid[nr][nc]);
```

```
        if (nextTime < dist[nr][nc]) {
            dist[nr][nc] = nextTime;
            pq.push({nextTime, nr, nc});
        }
```

```

        }
    }

    return -1;
}
};

```

16.4 Complexity

Time:

$O(n^2 \log n)$

Space:

$O(n^2)$

16.5 Alternative

Union Find by rising water time also works.

Binary search time + BFS also works.

17. Full Problem Walkthrough 6 — LeetCode 1334: Find the City With the Smallest Number of Neighbors

17.1 Problem Summary

Given weighted undirected graph and distance threshold.

Find city with smallest number of reachable cities within threshold.

If tie, choose greatest city index.

17.2 How to Identify Floyd-Warshall

Need all-pairs shortest paths.

n is usually small enough.

Use Floyd-Warshall.

17.3 C++ Code

```

class Solution {
public:
    int findTheCity(int n,
                    vector<vector<int>>& edges,
                    int distanceThreshold) {
        const int INF = 1e9;

        vector<vector<int>> dist(n, vector<int>(n, INF));

```

```

for (int i = 0; i < n; i++) {
    dist[i][i] = 0;
}

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];
    int w = edge[2];

    dist[u][v] = w;
    dist[v][u] = w;
}

for (int mid = 0; mid < n; mid++) {
    for (int start = 0; start < n; start++) {
        for (int end = 0; end < n; end++) {
            if (dist[start][mid] == INF || dist[mid][end] == INF) {
                continue;
            }

            dist[start][end] = min(dist[start][end],
                                   dist[start][mid] + dist[mid][end]);
        }
    }
}

int bestCity = -1;
int bestCount = INT_MAX;

for (int city = 0; city < n; city++) {
    int count = 0;

    for (int other = 0; other < n; other++) {
        if (other != city && dist[city][other] <= distanceThreshold) {
            count++;
        }
    }

    if (count <= bestCount) {
        bestCount = count;
        bestCity = city;
    }
}

```

```

        }
    }

    return bestCity;
}
};

```

17.4 Complexity

Time:

$O(n^3)$

Space:

$O(n^2)$

18. Full Problem Walkthrough 7 — LeetCode 1514: Path With Maximum Probability

18.1 Problem Summary

Edges have success probabilities.

Find path from start to end with maximum product probability.

18.2 How to Identify Dijkstra Variant

Instead of minimizing distance, maximize probability.

Use max-heap.

Relaxation:

```
newProb = prob[node] * edgeProb
```

If new probability is better, update.

18.3 C++ Code

```

class Solution {
public:
    double maxProbability(int n,
                        vector<vector<int>>& edges,
                        vector<double>& succProb,
                        int start,
                        int end) {
        vector<vector<pair<int, double>>> graph(n);

        for (int i = 0; i < edges.size(); i++) {

```

```

    int u = edges[i][0];
    int v = edges[i][1];
    double p = succProb[i];

    graph[u].push_back({v, p});
    graph[v].push_back({u, p});
}

vector<double> best(n, 0.0);
best[start] = 1.0;

priority_queue<pair<double, int>> pq;
pq.push({1.0, start});

while (!pq.empty()) {
    auto [prob, node] = pq.top();
    pq.pop();

    if (node == end) {
        return prob;
    }

    if (prob < best[node]) {
        continue;
    }

    for (auto [neighbor, edgeProb] : graph[node]) {
        double nextProb = prob * edgeProb;

        if (nextProb > best[neighbor]) {
            best[neighbor] = nextProb;
            pq.push({nextProb, neighbor});
        }
    }
}

return 0.0;
}
};

```

18.4 Complexity

Time:

$O((V + E) \log V)$

Space:

$O(V + E)$

19. Full Problem Walkthrough 8 — Min Cost to Connect All Points, Contrast With Shortest Path

19.1 Problem Summary

Connect all points with minimum total edge cost.

19.2 Important Distinction

This is **not** shortest path.

It asks for minimum cost to connect all nodes.

That is a Minimum Spanning Tree problem.

Use:

- Kruskal + Union Find
- Prim's algorithm

Do not use Dijkstra for shortest path from one source.

20. Full Problem Walkthrough 9 — Shortest Path in Binary Matrix, BFS Contrast

If grid moves all cost 1, use BFS.

Dijkstra would work but is unnecessary.

BFS is simpler:

all edge weights equal -> BFS

21. Full Problem Walkthrough 10 — Shortest Path in Grid With Obstacles, BFS State Contrast

If each move costs 1 but state includes remaining eliminations:

row, col, remaining k

Use BFS with state.

Do not use Dijkstra unless moves have different costs.

22. Mini Problem 1 — Count Number of Shortest Paths

When counting shortest paths in weighted graph, use Dijkstra plus ways array.

```
if (nextDist < dist[neighbor]) {
    dist[neighbor] = nextDist;
    ways[neighbor] = ways[node];
    pq.push({nextDist, neighbor});
} else if (nextDist == dist[neighbor]) {
    ways[neighbor] += ways[node];
}
```

Used in:

- Number of Ways to Arrive at Destination
-

23. Mini Problem 2 — Shortest Path in DAG

For DAG:

29. Topologically sort nodes.
30. Relax edges in topo order.

Works even with negative edges because there are no cycles.

```
for (int u : topoOrder) {
    if (dist[u] == INF) {
        continue;
    }

    for (auto [v, w] : graph[u]) {
        dist[v] = min(dist[v], dist[u] + w);
    }
}
```

24. Mini Problem 3 — Reconstruct Path After Dijkstra

```
vector<int> parent(n, -1);
```

```
if (nextDist < dist[v]) {
```

```
dist[v] = nextDist;
parent[v] = u;
}
```

After algorithm:

```
if (dist[target] == INF) {
    return {};
}
```

Build path:

```
vector<int> path;

for (int cur = target; cur != -1; cur = parent[cur]) {
    path.push_back(cur);
}

reverse(path.begin(), path.end());
```

25. Common Edge Cases for Shortest Path

25.1 Source Equals Target

Distance is usually 0.

But path length may be 1 if counting nodes.

25.2 Unreachable Target

Return:

-1

or empty path.

25.3 Multiple Edges Between Same Nodes

Keep minimum edge or allow relaxations to handle it.

25.4 Self Loops

Usually ignored unless beneficial.

25.5 1-Indexed Nodes

Use $n + 1$.

25.6 Large Distances

Use long long.

25.7 Negative Edges

Do not use Dijkstra.

25.8 Zero-Weight Edges

Dijkstra handles zero weights.

If only 0/1, 0-1 BFS may be faster.

26. Common Mistakes and Wrong Code

Mistake 1 — Using BFS on Weighted Graph

Wrong if weights differ.

Use Dijkstra or another weighted shortest path algorithm.

Mistake 2 — Using Dijkstra With Negative Weights

Dijkstra can produce incorrect answers.

Use Bellman-Ford.

Mistake 3 — Not Skipping Stale Heap Entries

Can cause extra work or incorrect early return in some variants.

Correct:

```
if (currentDist > dist[node]) {  
    continue;  
}
```

Mistake 4 — Wrong Stops Interpretation

In flights:

```
k stops = k + 1 edges
```

Loop $i \leq k$.

Mistake 5 — Overflow

Use long long when sums can exceed int.

Mistake 6 — Treating MST as Shortest Path

Connecting all nodes cheaply is MST, not shortest path.

27. Interview Explanation Templates

27.1 Dijkstra

Since all edge weights are non-negative, I use Dijkstra. I keep the best known distance to each node and process the node with smallest current distance using a min-heap. When I find a better route to a neighbor, I relax it and push it into the heap.

27.2 0-1 BFS

Since every edge cost is either 0 or 1, I use a deque. A 0-cost move goes to the front, and a 1-cost move goes to the back, preserving increasing distance order.

27.3 Bellman-Ford

Bellman-Ford repeatedly relaxes all edges. After at most $n - 1$ relaxations, shortest paths without negative cycles are found. It is useful when negative edges or edge-count limits exist.

27.4 Floyd-Warshall

I use Floyd-Warshall for all-pairs shortest paths. For each intermediate node, I try improving every pair of start/end nodes through that intermediate.

27.5 BFS vs Dijkstra

If every move has equal cost, BFS is enough. If costs differ but are non-negative, Dijkstra is needed.

28. Complexity Patterns

BFS	$O(V + E)$	$O(V)$	unweighted
Dijkstra	$O((V + E) \log V)$	$O(V + E)$	non-negative weights
0-1 BFS	$O(V + E)$	$O(V + E)$	weights 0/1
Bellman-Ford	$O(V * E)$	$O(V)$	negative edges
Floyd-Warshall	$O(V^3)$	$O(V^2)$	all pairs
DAG shortest path	$O(V + E)$	$O(V + E)$	DAG

29. Time-Boxed Practice Guidance

Intro Shortest Path

Target:

20–35 minutes

Examples:

- Network Delay Time
- Path With Maximum Probability
- Shortest Path in Binary Matrix

Medium Shortest Path

Target:

35–55 minutes

Examples:

- Path With Minimum Effort
- Cheapest Flights Within K Stops
- Minimum Cost Valid Path Grid
- Swim in Rising Water

Harder Shortest Path

Target:

60–90 minutes

Examples:

- Find the City
 - Shortest Path Visiting All Nodes
 - Number of Ways to Arrive at Destination
 - Minimum Obstacle Removal to Reach Corner
 - Race Car
-

30. Expanded Practice Set

30.1 Must-Do Medium

31. Network Delay Time
32. Path With Minimum Effort
33. Cheapest Flights Within K Stops
34. Minimum Cost to Make at Least One Valid Path in a Grid
35. Swim in Rising Water
36. Find the City With the Smallest Number of Neighbors
37. Path With Maximum Probability
38. Shortest Path in Binary Matrix
39. Shortest Path in a Grid with Obstacles Elimination
40. Number of Ways to Arrive at Destination

30.2 Strong Interview Practice

41. Minimum Obstacle Removal to Reach Corner
42. Reachable Nodes in Subdivided Graph
43. The Maze II
44. The Maze III
45. Cheapest Flights Within K Stops
46. Path With Maximum Probability
47. Minimum Cost to Reach Destination in Time
48. Shortest Path Visiting All Nodes

30.3 Hard / Stretch

49. Race Car
 50. Minimum Cost to Make at Least One Valid Path in a Grid
 51. Minimum Obstacle Removal to Reach Corner
 52. Count All Possible Routes
 53. Find Critical and Pseudo-Critical Edges in MST
 54. Modify Graph Edge Weights
 55. Second Minimum Time to Reach Destination
 56. Minimum Weighted Subgraph With Required Paths
-

31. Suggested Study Order

Step 1 — Separate BFS and Dijkstra

Start with:

57. Shortest Path in Binary Matrix
58. Network Delay Time
59. Path With Maximum Probability

Goal:
Know when BFS is enough and when Dijkstra is needed.

Step 2 — Dijkstra on Grid

Solve:
60. Path With Minimum Effort
61. Swim in Rising Water

Goal:
Learn non-sum path cost variants.

Step 3 — Special Weight Patterns

Solve:
62. Minimum Cost Valid Path Grid
63. Minimum Obstacle Removal

Goal:
Learn 0-1 BFS.

Step 4 — Edge-Limit Shortest Path

Solve:
64. Cheapest Flights Within K Stops

Goal:
Understand Bellman-Ford style relaxation.

Step 5 — All-Pairs Shortest Path

Solve:
65. Find the City With Smallest Neighbors

Goal:
Learn Floyd-Warshall.

32. Pattern Decision Table

minimum steps, unweighted	BFS
weighted positive costs	Dijkstra
edge cost is 0 or 1	0-1 BFS
negative weights	Bellman-Ford
at most K stops/edges	Bellman-Ford style or BFS state
all-pairs shortest path	Floyd-Warshall
grid with equal move cost	BFS
grid with varying cost	Dijkstra / 0-1 BFS

Problem Clue	Algorithm
maximize probability	Dijkstra max-heap
minimize maximum edge	Dijkstra variant / binary search
connect all nodes min total	MST, not shortest path

33. Final Cheat Sheet

Dijkstra Relaxation

```
if (dist[u] + w < dist[v]) {
    dist[v] = dist[u] + w;
    pq.push({dist[v], v});
}
```

Skip Stale Entry

```
if (currentDist > dist[node]) {
    continue;
}
```

0-1 BFS Push Rule

```
if (weight == 0) {
    dq.push_front(neighbor);
} else {
    dq.push_back(neighbor);
}
```

Bellman-Ford Relaxation

```
for (int i = 0; i < n - 1; i++) {
    for (edge : edges) {
        relax(edge);
    }
}
```

Floyd-Warshall

```
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}
```

34. What to Master Before Chapter 30

Before moving to Minimum Spanning Tree, make sure you can:

- Identify shortest path problems
 - Choose BFS vs Dijkstra
 - Implement Dijkstra with min-heap
 - Skip stale heap entries
 - Build weighted adjacency list
 - Use Dijkstra on grid
 - Handle non-sum path costs like max edge
 - Implement 0-1 BFS
 - Understand Bellman-Ford relaxation
 - Understand Floyd-Warshall
 - Know why Dijkstra fails with negative weights
 - Solve Network Delay Time
 - Solve Path With Minimum Effort
 - Solve Cheapest Flights Within K Stops
 - Solve Minimum Cost Valid Path Grid
 - Distinguish shortest path from MST
-

35. Core Takeaway

Shortest path is about minimum cost from one place to another.

The correct algorithm depends on edge weights:

unweighted -> BFS
non-negative weighted -> Dijkstra
0/1 weights -> 0-1 BFS
negative weights -> Bellman-Ford
all pairs -> Floyd-Warshall
DAG -> topological DP

The most common interview mistake is using BFS when edge costs differ.

The second most common mistake is using Dijkstra when negative weights exist.

Always ask:

What are the edge weights?

That answer determines the algorithm.
